SAMPLE

Building the Firmware Infrastructure of Tomorrow

Building the Invisible Half of Every Machine

Aiman JoeHanz

Free sample: Introduction & Chapter 1

putrabooks.github.io

Introduction

Stand in the kitchen at midnight with the lights off and listen to the infrastructure at work: the refrigerator's compressor deciding its cycle, the dishwasher remembering its paused program, the thermostat sampling the hallway's air, the router's pilot lights counting the neighborhood's packets, the television waiting on its stand-by promise, the smoke detector's slow green blink, the phone charging to a negotiated current, the car in the drive holding eleven networks awake while the household sleeps. Not one of those machines is thinking with a program the owner installed, and almost none of their code will ever be seen, updated, or named by anyone who owns them. Every one of them runs on firmware: the software poured into hardware at the factory, the invisible half of every machine that ships without a screen, a keyboard, or an apology. This book is about that half, and about the quiet global discipline that writes, tests, signs, ships, and repairs the code inside things.

Firmware deserves the infrastructure treatment precisely because its success is measured in being forgotten. The washing machine that runs a decade without a reboot, the meter that reports for thirty years, the pacemaker that never asks for attention, the vehicle module that outlives the vehicle: these are not features anyone shops for, but they are the promises every embedded device quietly makes, and behind each promise stands the same anatomy the reader will walk in these chapters. The promise can be written exactly: the code starts every time power arrives, it finishes its work within deadlines a human can't perceive, it keeps its secrets, it survives its own failures, and it can be replaced in the field without bricking the machine. The machinery for keeping those promises is what this book opens, layer by layer, from the first electric breath of a powered chip to the supply chain that will be asked to sustain it for decades.

The scale of the subject is easy to miss because its units are so small. A single embedded project may fit in a few hundred kilobytes, less text than one chapter of this book, yet govern a valve that moves a city's gas or a sensor that guards a refinery. Billions of such devices ship every year, in thermostats and tractors, meters and medicine, toys and trains, and the world's fleet of running firmware now dwarfs the population of phones and laptops combined. The engineers who write it work under constraints the desktop world forgot decades ago: kilobytes instead of gigabytes, milliwatts instead of watts, deadlines measured in microseconds, hardware that cannot be patched by hand and must be right the first time or dead forever. Those constraints are not obstacles to the discipline; they are the discipline, and this book treats them as the anatomy's first principle.

The tour follows the life of the code rather than the layout of the textbook. First the landscape of embedded systems, what firmware is and where it lives. Then the boot, the first breath of a powered device. Then the toolchain, the translation from human text to machine bytes. Then the runtime, the code's daily life of interrupts and deadlines. Then the driver layer, where software learns to speak hardware's narrow dialects. Then the update, the most dangerous operation in the machine's life. Then the debug, the craft of finding what the silicon hides. Then the security, the code's armor in a world of physical attackers.

Then the rules, the paperwork regimes by which strangers extend trust to code they cannot read. And finally the long run, where a product built in one decade must be maintained into the next. Every chapter keeps the same habit the series always keeps: no product names, no version numbers, no vendor dialects, just the working anatomy of the machine, written plainly enough to argue with.

Chapter 1: The Embedded Landscape — Where Firmware Lives

Begin with the definition that fits in one sentence and then pull it apart. Firmware is software that is part of the machine: not installed on the machine but resident in it, poured into flash memory at manufacture, started automatically by hardware, and expected to run for the machine's whole life without supervision. The sentence hides the real subject, which is constraint. The desktop programmer inherits a world of gigabytes and gigahertz and asks the operating system for resources; the firmware engineer inherits a budget of kilobytes, a clock measured in milliwatts of permission, and no operating system at all, or a sliver of one, and must still deliver the same reliability the public expects from a kitchen faucet. That inversion, luxury traded away for permanence, is what makes the embedded world its own discipline, and every chapter of this book is a working out of that one trade.

The census of devices is the landscape's map, and it is far larger than the phone-shaped tip most people see. The factory floor: the drives and the controllers, the sensors sampling vibration a thousand times a second, the safety relay that must trip in milliseconds. The grid: the meters and the breakers, the inverter governing a solar roof, the charger negotiating with a car. The vehicle: dozens of communicating modules, each a small computer with its own

firmware and its own deadlines, from the mirror's heater to the bag's controller. The home: the appliance, the thermostat, the lock, the camera. The body: the pacemaker, the insulin pump, the hearing aid, each running code that must not fail in a lifetime. The public infrastructure: the traffic signal, the

elevator, the water valve, the payment terminal. The census teaches the scale truth: the world runs on the small machines, and the smaller the machine, the longer its code must work unattended.

The firmware stack itself arranges in layers the rest of the book will walk in order, and the reader should see the whole staircase once before climbing it. At the bottom, the hardware: the processor core, the memory, the peripherals, the pins. Above it, the boot layer: the small code that decides whether the machine will start and what it will start. Above that, the runtime: the initialization, the interrupt handlers, the scheduler that shares the single processor among its many duties. Above that, the drivers: the translation layer where the code learns each peripheral's dialect. Above that, the application: the state machines and the protocols, the logic the buyer actually experiences. And threaded through all of it, written vertically rather than in layers: the debug, the security, the testing, and the release engineering, the disciplines that make the stack trustworthy rather than merely working. The stack is thin, often a few hundred kilobytes in total, but every layer is load-bearing.

The people of the landscape deserve their census too, because the machine's promise is kept by a guild the public never meets. The electrical engineer who speaks the schematic and is only half joking when calling the software the last component; the embedded software engineer who reads register maps like literature and counts bytes like money; the toolchain specialist and the build engineer who own the pipeline from text to binary; the test engineer who builds the rigs that abuse prototypes on purpose; the security engineer who assumes the attacker has a screwdriver and an oscilloscope; the field engineer who reads crash logs from a machine ten time zones away; and the program manager who holds the release together against the deadline's pull. The chapter's closing thought sets up the whole book: every one of those people works on code that cannot be seen, in machines that cannot be opened, for users who will never know they exist, and the craft's central problem is making the invisible trustworthy.

The vocabulary lesson comes early in the landscape chapter because the words themselves mislead. Embedded, the adjective for the whole discipline, named for code that is resident inside the product rather than installed onto it, and hiding in that word the deeper truth: the code is embedded in the machine's identity, its warranty, its certification, and its liability in a way no desktop software ever is. Real-time, which does not mean fast but means on time: the machine's answer is worth nothing after its deadline passes, whether the deadline is a millisecond or a second. Firmware, a word the industry half-abandoned and half-kept, the resident code in its own memory, sometimes a few kilobytes, occasionally a full operating system's worth. Hard real-time

against soft: the brake that must never miss against the display that may drop a frame. The reader who learns these five words precisely has learned the embedded landscape's map legend, and the rest of the book will use them honestly.

The landscape's economics deserve their paragraph, because the scale that governs the industry is not the unit's price but the unit's fleet. The five-cent microcontroller that ships in a hundred million toasters, where a one-byte improvement saves more memory than some desktop products own and a one-cent cost saving is a fortune; the five-hundred-dollar module in the thousand-unit medical device, where the engineering hour is cheap against the certification hour and the code is polished like an instrument. Between them every economy in the trade: the consumer product engineered in months and priced to the cent, the industrial controller engineered for decades and priced to the promise, and the shared truth that in every case the software's cost is mostly paid after the release, in the maintenance the long run chapter counts.

The landscape's lesson for the reader is that embedded engineering is above all an economics of constraints, and the constraint that governs the design is whichever of pennies, bytes, watts, or years the product's market has chosen to be scarce.

The landscape's history belongs here briefly, because the field's age explains its temperament. The first embedded projects were aerospace and telecom: the guidance computers that had to work with the memory of a modern greeting card, the telephone switches that could never go down, the disciplines of proof and parsimony born where failure was a lost mission or a silent city. The microcontroller's democratization then moved the discipline into every product: the appliance, the toy, the car, the meter, and the trade's old rigor met the consumer's new price point, producing the hybrid culture the reader meets today, where a team may ship a safety-certified brake controller on Tuesday and a singing greeting card on Wednesday. No infrastructure in this series has a shorter history or a longer reach, and none inherited stricter habits from its ancestors, which the certification chapter will show in full.

The landscape's reader deserves the chapter's closing orientation, the question the census and the stack and the professions all serve: what makes firmware an infrastructure rather than a craft novelty, and the answer is the fleet.

One device is a gadget; a hundred million devices with a common binary are a public system, running on the same code in hospitals and homes, in cars and grids, owned by people who never consented to be the code's beta testers.

The fleet changes the engineering obligations: the update machinery of the later chapters, the telemetry of the debug chapter, the security maintenance of the armor chapter exist because the fleet, not the device, is what the manufacturer really ships. The reader who holds the fleet in mind will understand every design decision in the chapters that follow, because the fleet is the reason the discipline is paranoid.

The bill of materials for the reader's own home deserves a paragraph of census practice, because the landscape is best learned on one's own shelf. Count the firmware in a single ordinary room: the television and its stand-by code, the router and its operating lifetime of packet service, the thermostat's control loop, the smoke detector's decade-long duty cycle, the doorbell's radio stack, the washing machine's drum sequencer, the vacuum's motor governor, the power tool's battery management, the car key's immobilizer exchange, the toy's sound synthesizer, the scale's load-cell sampling, the printer's whole hidden computer more powerful than the desktops of the design's childhood. The count passes thirty without effort, crosses a hundred in a full household, and the reader's street runs to the millions. The census practice's point is not the total but the texture: each of those devices was certified, shipped, forgotten, and is still keeping its promise tonight, which is the landscape's ordinary miracle, performed by the guild these chapters introduce.

The simulation's role in the modern landscape deserves the paragraph, because the industry has learned to ship products before the hardware exists. The instruction-set simulator: the desktop that executes the embedded binary faithfully enough to develop the application while the board is still a schematic; the peripheral's model: the timer and the converter imitated in software so the driver can be written against a pretend machine; the factory's full simulation: the plant, the vehicle, the body modeled well enough that the certification evidence itself can be gathered partly in the modeled world. The honest limits: the simulation never quite matches the analog edge, the timing never quite the silicon's truth, and the first hardware bring-up always finds what the model forgave. The landscape's modern shape: the simulator has become the embedded team's primary development machine, with the real board arriving as a late and humbling stage.

The safety margin between working and safe deserves the landscape's engineering paragraph, because the embedded world's most important distinction is invisible to the desktop eye. The machine that works: the code runs, the feature ships, the demo glows; the machine that is safe: the same code, when the sensor lies, when the brownout strikes, when the operator's toddler holds every button at once, degrades into a state that harms nothing. The disciplines that separate the two: the defensive checks that treat every

input as hostile, the watchdog and the fault containment of the runtime chapter, the failsafe design where the default state is the harmless one, and the hazard analysis of the rules chapter performed even on the products no regime requires, because the guild's habit travels with its members.

The reader should carry the landscape's sternest sentence into every later chapter: in the embedded world, working is not the finish line; the finish line is failing well, and everything in this book is preparation for the reader's own product's worst afternoon, which will arrive unannounced, as it always does, at the least convenient hour the calendar could choose.

The landscape's unseen majority deserves the census's final expansion: the machines that are not products at all but components inside products. The motor controller buried inside the appliance, the battery governor inside the tool, the airbag's decision box behind the dashboard, the meter's measuring die inside the meter's shell: the firmware inside the firmware's customer, sold business to business, invisible even to the product's own teardown photographers. This majority changes the industry's texture: the buyer is an engineer with a specification and a qualification rig, the seller a supplier with an errata history and a support obligation measured in decades, the negotiation conducted in reference manuals rather than marketing pages.

The reader's census instrument for this stratum: open any appliance and find the two or three branded modules with their own silkscreened names, each the visible tip of an engineering firm whose entire business is the invisible layer, the true silent majority of the embedded landscape, and the economic foundation on which the visible consumer layer quietly stands, forever unnamed in the store's aisle, the landscape's deepest lesson in one glance.

The book's method deserves its own closing paragraph in the landscape chapter, because the reader should know how the rest of the tour will be conducted and what it deliberately omits. The tour walks layers in the machine's own order of life: the boot before the toolchain that builds it in the reader's attention, because the reader is following the code's biography rather than the curriculum's convenience; the driver layer before the security, because the armor is meaningless until the reader knows what is being armored; the rules before the long run, because the certificates are the promises the long run must keep. What the tour omits on purpose: every product

name, every version number, every benchmark, every trend that will date these pages within a season, the whole ephemeral layer that the industry's marketing maintains and this series refuses. The reader who wants today's headlines has a whole internet of them; the reader who wants the anatomy

that will still be true when the reader's current devices are landfill has these ten chapters, and the book's method is the promise that the anatomy, described plainly, outlasts the catalog, which the next nine chapters now begin to prove.

The interface's poverty deserves the landscape's craft paragraph, because the embedded machine's user experience is built from almost nothing. The palette: a single LED in its patterns, a button whose press and hold and double-click must carry a whole grammar, a seven-segment's terse numerals, a single beep's vocabulary of meanings, the display of a few characters whose every line was argued over; the discipline: each pattern documented in the manual and mirrored in the firmware's own constants, each meaning distinct enough to survive a translation, each fault legible enough that the support call begins with the right question.

The reader who has stared at a blinking light wondering what it meant has met the discipline's frontier: the machine's only voice is a design budget of pennies and pins, and the firmware author is also, whether the job description admits it or not, the product's playwright, writing a drama in one light's tempo, for an audience that must understand the performance without a program, at midnight, in the reader's dark kitchen, holding the manual's last page.

The landscape's failure imagination deserves the final expansion, the discipline of asking what the machine will do at its worst. The practice, borrowed from the safety world and deserved by every product: the premortem, the team gathering before the design hardens to write the field report of the failure that has not happened yet, the angry customer's letter composed in advance, the warranty claim's anatomy imagined, and the design then changed to make the imagined letter unprintable; the questions the practice asks: what happens when the power fails mid-write, when the sensor lies, when the update's server is gone, when the customer holds the button for a year, when the sibling chip's errata is real, when the installer reverses the polarity the manual never covered.

The reader should treat the failure imagination as the landscape chapter's graduation exercise: the embedded engineer's defining habit is thinking in failure first and features second, and every chapter that follows is that habit given a machinery, a checklist, and a certificate, the whole book being, in the end, a sustained act of pessimism performed in the service of trust.

The prototype's journey deserves the landscape's manufacture paragraph, because the code's path from the bench to the fleet passes through a factory most readers never see. The engineering builds: the hand-assembled boards with their rework wires and their debug ports open, the firmware loaded by the

engineer's own probe, the units the team signs out like library books; the preproduction builds: the pilot line's first hundred units, the firmware's bugs now visible in the factory's tempo, the programmer's fixture and its golden image, the calibration's station and its records; the production builds: the thousandth unit's binary built by the reproducible toolchain and verified by the line's own checks, the same bits the desk signed now traveling to the customer's home.

The journey's disciplines, the reader should notice, are all in this book's earlier chapters: the toolchain's reproducibility, the update's verification grammar, the identity's provisioning ceremony, because the factory is where those disciplines pay their first dividend, and the manufacturer's whole reputation rides on the line's humble routine, repeated millions of times, without an engineer in the room.

The cost of bytes deserves the landscape's closing economics, the arithmetic that makes the embedded trade so tight. The math the industry lives by: a feature's memory cost multiplied by the fleet becomes the next chip's size class, the difference between the part that fits and the part that does not, priced in the millions per year; a feature's cycle cost multiplied by the fleet becomes the battery's chemistry, the supply's voltage, the heat sink's gram weight; a feature's engineering cost divided by the schedule becomes the certification's delay, the quarter's miss, the market's window.

The discipline this arithmetic breeds: the byte counted like a coin, the feature argued like a budget item, the library's adoption weighed like a hire, and the veteran's instinct, honed across projects, for what the small machine can afford. The reader should hold the landscape's sternest lesson before the tour descends into the layers: nothing in the embedded world is free, everything is priced in the fleet's millions, and the pricing is the design, which the next nine chapters now demonstrate, layer by layer, at the meter's own tempo, in the currency of bytes, watts, and years.

The reader's own position deserves the landscape chapter's final paragraph, because the book's first layer ends where the series always aims: the reader's own kitchen. The census conducted in the reader's home produced a number in the hundreds of quietly working machines; the anatomy about to be opened explains every one of them: the boot when the reader plugs the device in tonight, the interrupts being served in the wall's wiring, the driver's dialect at the meter's dial, the update's campaign somewhere in the reader's neighborhood, the security's chain

holding against attackers the reader will never meet, the certificates behind the appliances the reader's family touches. The reader's new standing, one

chapter into the book: the rare person who knows the question to ask, and the next nine chapters exist so the reader can also understand the answer, at whatever depth the reader's own work and household require, which is the series' whole contract with its readers, stated here plainly at the first layer's end, and honored at the last page, as always.

The field's weather deserves the landscape's reliability paragraph, the environment the fleet actually lives in. The cabinet's honesty: the heat of the summer's peak with the door closed, the cold of the delivery truck, the humidity's condensation on the device unboxed at dawn, the dust of the mill and the salt of the harbor, the vibration of the pump and the altitude of the pipeline; the electronics' answers: the thermal design's margins audited at the worst ambient rather than the lab's comfort, the conformal coat and the sealed enclosure's arithmetic, the connector's retention tested against the fleet's roads.

The firmware's contribution, the paragraph's point: the machine's survival is code as much as hardware, the brownout's tolerance, the watchdog's honesty, the sensor's fusion discounting the temperature's drift, the failure's safe state in the environment that broke the sensor, and the whole reliability's evidence gathered in the endurance lab's campaign: the thousand-hour bake, the thermal shock's cycles, the humidity's swamp, the results filed as the design's promise. The reader should hold the weather paragraph's sternness: the fleet lives outdoors, in attics, in factories, in pockets, in all the world's actual conditions, and the design that was proven only on the bench has been proven nowhere, at the planet's own tempo, which has no bench at all.

The landscape's organizational honesty deserves the closing expansion: who in the company owns the invisible half. The startup's reality: the firmware written by the electrical engineer, the hardware's author, because someone must, the discipline's whole curriculum compressed into one person's evenings; the growing firm's middle age: the embedded team formed, the boundaries argued, the hardware author who believes software is a component and the software author who believes hardware is a nuisance, the interface's discipline negotiated at the schematic's edge; the mature firm's settlement: the systems approach, the boot and the security owned as products, the toolchain staffed, the telemetry staffed, the long-run's maintenance funded as a line item rather than a hope.

The reader should see the landscape's last truth: the machine's quality follows the org chart, because the code that nobody owns is the code that ships by accident, and the guild's disciplines, all ten chapters of them, are the practices that only survive when someone is paid to keep them, the whole book being,

at its widest angle, a manual of ownership, in the most literal sense the word has.

The naming of things deserves the landscape's final craft paragraph, because the embedded world's software is a system of names the public never hears and the engineers live inside. The project's code names and the silicon's part numbers, the peripheral's register mnemonics, the bus's standard's letter-salads, the errata's clause numbers that veterans quote like scripture verses: a private language, half of it inherited from datasheets, half invented at whiteboards, and all of it load-bearing, because the fleet's engineers a decade apart must mean the same thing by the same word. The discipline the mature team practices: the glossary maintained as a living artifact, the names chosen for the engineer who arrives in five years rather than the joke that lands in today's standup, the hardware revision's letters used consistently from the schematic to the telemetry to the support script.

The reader should see the landscape's last census: the invisible machine's most invisible layer is its language, and the guild's quality is legible in its vocabulary long before any code is read, because a team that names things carefully tends to design them carefully, the correlation being one of the trade's oldest observations, and one this book's own chapters have quietly tried to deserve, at every layer, in every sentence.

The landscape's brother trades deserve the final paragraph of the first chapter, the disciplines around the embedded table. The hardware author, the schematic's poet, whose patience the whole book depends on: the boot's first code runs only because someone chose the rails and the reset's timing with the software's needs in mind; the test author, whose rig the rules chapter met, whose automated abuse of the prototype is the only reason the fleet's ordinary evening is possible; the reliability engineer, whose chamber's endurance campaign writes the warranty's fine print; the manufacturing engineer, whose line the toolchain's binaries travel every hour; and the product's own voice, the support engineer, who reads the field's worst days and files them as the design's teachers.

The first chapter's census closes with the whole table in view, because the ten chapters that follow are actually ten chairs at this same table, each chapter a guild of its own, and the reader who has finished the landscape now knows the industry's true shape: not a pyramid of products but a circle of crafts, each holding a piece of the promise, and the invisible machine the sum of their hands, which the rest of the book now opens, chair by chair, layer by layer, to the last page.

The landscape's moral census deserves the chapter's final expansion: which

machines deserve which care, and who decides. The spectrum's honesty: the toy's firmware that may crash and be thrown away without consequence beside the pump's firmware that may not fail in a lifetime, and between them the whole gradation of stakes, the appliance that disappoints, the lock that embarrasses, the meter that bills, the vehicle that carries, the body's own implant that is the person; the decision's location, the paragraph's point: the gradation is decided by engineering management, in the requirements meeting, long before the certification's regime is chosen, and the guild's recurring failure is the mismatch, the cheap toy engineered like the pump and the pump shipped with the toy's care, both expensive in their own directions.

The reader's instrument, the landscape's last gift: the question what happens when this fails, asked at the design's start, is the entire embedded discipline in nine words, and the ten chapters of this book are simply the machinery different answers require, which the reader now holds, in order, for the rest of the tour, and for the reader's own products, whatever their stakes turn out to be.

The bill of the invisible deserves a closing economics note in the landscape chapter, the arithmetic of what the world spends on what it never sees. The industry's honest scale: the embedded silicon's trade now rivals the visible computer industry it quietly dwarfs in units, the firmware engineering embedded in every product's cost basis, the maintenance decades priced into the warranty's reserve, the certification's fees folded into the medical device's and the vehicle's margins, and the whole global bill arriving to the customer as pennies inside products, invisible, uncontested, and enormous; the market's blind spot, the paragraph's caution: the buyer who cannot see the firmware cannot comparison-shop it, and the manufacturer's temptation, always, is the invisible cost's quiet erosion, the cheaper part, the thinner test, the shorter support window, because nobody sees the difference at the shelf.

The reader, one chapter in, already sees it: the update's policy, the parts window, the telemetry's candor are the firmware's visible price tags, and the rest of this book teaches the reader to read them, which is the market's own correction, arriving one informed customer at a time.

The tour's ticket deserves the landscape's true final paragraph, the reader's orientation for the nine chapters ahead, written as a promise. The chapters will descend and ascend: the boot's milliseconds, the toolchain's translation, the runtime's deadlines, the driver's dialects, the update's surgery, the debug's forensics, the security's armor, the rules' certificates, the long run's decades, each layer opened with the series' standard instruments, the exact promise named first and the machinery audited second, no vendors named and no

versions quoted, the anatomy written to outlast the catalog as always; the reader's effort, honestly priced: no mathematics beyond arithmetic, no code required to be read, the machine's habits explained in the plain prose the trade's best engineers use when they teach.

The landscape chapter's last sentence hands over the ticket: the reader who has finished this census of devices, people, stacks, and stakes is equipped for everything that follows, and the next chapter begins, as the machine does, at the very first instruction, in the dark, at power's arrival, where the whole book, like the whole fleet, now starts.

The counterfactual tour deserves one more landscape paragraph, the thought experiment that calibrates the discipline's stakes: a day the firmware failed everywhere at once. The morning the household's machines woke confused: the thermostat that forgot its schedule, the appliance that refused its program, the meter that lost its count, the lock that forgot its owner, the vehicle that asked for its service, the pump that paused its dose; the engineers' estimate, the paragraph's quiet point: almost none of those failures would be the hardware's, because silicon ages slowly and rarely, and almost all of them would be the invisible half, the code's boundary conditions meeting the world's edge cases, which is why the trade's paranoia is not a temperament but a correct response to its actual position: the fleet's reliability, in the plain arithmetic of its root causes, is mostly a software property, and the public's trust in its machines is mostly a trust in people it has never met, writing code it will never see, for mornings it is determined to prevent, one disciplined chapter at a time, which this book has now opened, and the tour descends.

The shelf's futures deserve the landscape's very last paragraph: what happens to this book's subject in the years after these pages are printed, written with the discipline's usual humility about prophecy. The fleets will grow, the machines will shrink, the two trends meeting in the embedded world's oldest arithmetic: more units, less per unit, the same promises per unit, forever. The disciplines will converge: the security's armor and the safety's certificates already meet in the vehicle, the ingredient list and the telemetry already meet in the regulation, and the guild's crafts, this book's ten chapters, will keep merging into the unified maintenance culture the connected fleet demands.

The reader who met the landscape's census with surprise will meet the future's census with fluency, and the fluency, not the prophecy, was the purchase: the anatomy ages slowly because the constraints age slowly, and the reader's working sight, once acquired, keeps updating itself at the market's own tempo, which is the series' whole design, honored here one last time at the first chapter's final margin, before the boot begins.

End of free sample. The full book is available at putrabooks.github.io